



COMANDOS DE CONSOLA

Guía completa de línea de comandos en Linux

SINTAXIS

COMANDOS

STREAMS

<> SINTAXIS DE UNA LÍNEA DE COMANDO

ESTRUCTURA BÁSICA

Una línea de comando sigue una estructura lógica que permite controlar procesos y gestionar datos de entrada/salida.

01 COMANDO

Programa o utilidad a ejecutar

02 OPCIONES

Flags que modifican el comportamiento (-l, -a, --help)

03 ARGUMENTOS

Archivos, directorios o datos de entrada

EJEMPLO VISUAL

```
usuario@linux:~$ ls -la /home/user > archivo.txt
```

REDIRECCIONES

< > >> Controlan flujos de E/S

PIPES

| Conectan salida de un comando a entrada de otro

ENCADENAMIENTO

; && || Ejecutan comandos en secuencia

CARACTERES ESPECIALES

* ? [] ~ Comodines y expansiones

COMANDOS INTERNOS Y EXTERNOS

COMANDOS INTERNOS

Integrados en el shell. No generan nuevos procesos. Ejecución más rápida.

EJEMPLOS

```
cd /home/user
echo "Hola Mundo"
type cd
history
alias
```

Verificación: type cd → cd is a shell builtin

COMANDOS EXTERNOS

Programas independientes ubicados en /bin, /usr/bin, etc. Generan nuevos procesos.

EJEMPLOS

```
ls -la /home
cat /etc/passwd
grep "root" /etc/passwd
sed 's/old/new/g' archivo
find . -name "*.txt"
```

Verificación: type ls → ls is /usr/bin/ls

COMANDOS BÁSICOS: NAVEGACIÓN Y VISUALIZACIÓN

LS

Lista contenido de directorios

```
ls -la /home
ls -lh /var/log
```

CD

Cambia de directorio de trabajo

```
cd /etc
cd ..
cd ~
```

CAT

Muestra contenido completo de archivos

```
cat /etc/hostname
cat /etc/os-release
```

HEAD

Muestra las primeras líneas de un archivo

```
head -n 10 /etc/passwd
head -n 5 /var/log/syslog
```

TAIL

Muestra las últimas líneas de un archivo

```
tail -n 20 /var/log/syslog
tail -f /var/log/auth.log
```

TIP

tail -f monitorea cambios en tiempo real

head/tail -n controla cantidad de líneas

T COMANDOS DE PROCESAMIENTO DE TEXTO

↑↓ TAC

Invierte el orden de las líneas

```
tac /etc/hosts  
cat file.txt | tac
```

<> STRINGS

Extrae texto legible de archivos binarios

```
strings /usr/bin/ls  
strings -n 10 archivo.bin
```

⌵ WC

Cuenta líneas, palabras y caracteres

```
wc -l /etc/passwd # Líneas  
wc -w /var/log/syslog # Palabras
```

✎ SED

Editor de flujo para transformaciones de texto

```
sed 's/old/new/g' archivo.txt  
sed '2d' /etc/hosts # Elimina línea 2  
sed '-n 5,10p' archivo # Líneas 5-10  
sed '/^#/d' config # Borra comentarios
```

💡 CONSEJOS ÚTILES

- **tac** es útil para logs en orden cronológico inverso
- **strings** encuentra mensajes en ejecutables y librerías
- **sed -i** modifica archivos en su lugar (con cuidado)
- **wc -l** es ideal para contar registros en archivos de datos

GREP: BÚSQUEDA DE PATRONES

GREP

Busca patrones en archivos de texto

```
grep "root" /etc/passwd
grep "error" /var/log/syslog
```

OPCIONES CLAVE

- i** Ignora mayúsculas/minúsculas
- r** Búsqueda recursiva en directorios
- n** Muestra números de línea
- v** Invierte la coincidencia (no contiene)
- c** Solo cuenta coincidencias

EJEMPLOS PRÁCTICOS

```
grep -i "localhost" /etc/hosts
grep -rn "sudo" /etc/
grep -v "^#" /etc/ssh/sshd_config
grep -c "error" /var/log/syslog
grep -E "root|admin" /etc/passwd
```

CASOS DE USO

- **grep -i** busca sin importar mayúsculas
- **grep -rn** localiza texto en proyectos
- **grep -v "^#"** filtra comentarios en configs
- **grep -E** usa expresiones regulares extendidas



EXPRESIONES REGULARES CON GREP

QUÉ SON

Patrones de búsqueda que describen conjuntos de cadenas mediante caracteres especiales

<> PATRONES BÁSICOS

^ Inicio de línea

```
grep "^root" /etc/passwd
```

\$ Final de línea

```
grep "bash$" /etc/passwd
```

. Cualquier carácter

```
grep "r..t" /etc/passwd
```

***** Repetición (0+)

```
grep "ab*c" archivo
```

[abc] Conjunto de caracteres

```
grep "[abc]pp" /usr/bin
```

[^abc] Negación

```
grep "[^#]" config
```

EJEMPLOS PRÁCTICOS

```
grep -E "^root|^nobody" /etc/passwd
grep -E "[0-9]{1,3}\.[0-9]{1,3}" /etc/hosts
grep -E "bash$|sh$" /etc/passwd
grep -E "^(#|$)" config # Comentarios/vacías
grep -vE "^#|^$" /etc/ssh/sshd_config
grep -E "[A-Z]{3,}" archivo # Mayúsculas consecutivas
```

PATRONES AVANZADOS

- **[a-z]** - Rango de letras minúsculas
- **{n,m}** - Repetición entre n y m veces
- **+** - Una o más repeticiones (grep -E)
- **?** - Cero o una repetición (grep -E)
- **|** - Operador OR (grep -E)
- **\b** - Límite de palabra

⚙️ DESCRIPTORES DE FICHEROS: PROCESOS

⚙️ ¿QUÉ ES UN PROCESO?

Instancia en ejecución de un programa. Cada proceso tiene su propio espacio de memoria y descriptores de ficheros.

📁 DIRECTORIO /PROC

Sistema de archivos virtual que contiene información sobre procesos en ejecución

```
/proc/[PID]/ - Info del proceso  
/proc/[PID]/fd/ - Descriptores abiertos  
/proc/[PID]/status - Estado del proceso
```

💡 CONSULTAR DESCRIPTORES

```
ls -la /proc/$$/fd  
lsof -p [PID]
```

🔗 DESCRIPTORES POR DEFECTO

0

stdin

Entrada estándar

1

stdout

Salida estándar

2

stderr

Error estándar

📋 CONTENIDO DE /PROC/[PID]/FD

```
0 -> /dev/pts/0 # Terminal actual  
1 -> /dev/pts/0 # Misma terminal  
2 -> /dev/pts/0 # Misma terminal  
3 -> socket:[12345] # Socket abierto  
4 -> /home/user/file.txt # Archivo abierto
```

ENTRADA/SALIDA ESTÁNDAR Y ERROR ESTÁNDAR

0

STDIN Entrada Estándar

Flujo de datos de entrada. Por defecto: teclado

```
/dev/stdin -> /proc/self/fd/0
```

1

STDOUT Salida Estándar

Flujo de datos de salida normal. Por defecto: pantalla

```
/dev/stdout -> /proc/self/fd/1
```

2

STDERR Error Estándar

Flujo de mensajes de error. Por defecto: pantalla

```
/dev/stderr -> /proc/self/fd/2
```

EJEMPLOS PRÁCTICOS

```
echo "Hola" # -> stdout (1)
ls /noexiste # -> stderr (2)
cat < archivo.txt # <- stdin (0)
grep "patrón" archivos* # stdin -> stdout
find / -name "*.log" 2>/dev/null
```

CONCEPTOS CLAVE

- **Separación de flujos:** stdout y stderr son independientes
- **Redirección selectiva:** cada flujo puede ir a destinos diferentes
- **Pipes:** conectan stdout de un comando a stdin de otro
- **Descriptores 3-9:** disponibles para archivos abiertos
- **Herencia:** procesos hijos heredan descriptores del padre

↔ REDIRECCIONES DE DESCRIPTORES

📁 REDIRECCIÓN DE ENTRADA

<

stdin desde archivo

```
cat < input.txt
```

<<<

here-string

```
cat <<< "texto"
```

📁 REDIRECCIÓN DE SALIDA

>

stdout a archivo (sobrescribir)

```
echo "texto" > out.txt
```

>>

stdout a archivo (añadir)

```
echo "más" >> out.txt
```

❗ REDIRECCIÓN DE ERROR

2>

stderr a archivo

```
cmd 2> err.txt
```

2>&1

stderr a stdout

```
cmd > out 2>&1
```

&>

stdout+stderr

```
cmd &> all.txt
```

📁 EJEMPLOS PRÁCTICOS

```
sort < datos.txt > ordenado.txt
ls -la 2>/dev/null # Ignorar errores
find / -name "*.log" 2>errores.log
cat <<< "Hola Mundo" # Here-string
grep "patrón" archivo &> resultado.txt
cmd1 2>&1 | cmd2 # Pipe con stderr
```

💡 CASOS DE USO COMUNES

- **2>/dev/null**: Descartar mensajes de error
- **&> archivo**: Capturar toda la salida para debugging
- **>> archivo.log**: Añadir a logs existentes
- **2>&1**: Combinar flujos para procesamiento
- **<<<**: Pasar cadenas como entrada
- **cmd > out 2> err**: Separar stdout y stderr

... OTROS SÍMBOLOS EN LÍNEA DE COMANDO

🔗 PIPE Y SECUENCIA

|

Conecta salida a entrada

```
ls | grep "txt"
```

;

Ejecutar en secuencia

```
cd /tmp; ls
```

🧠 OPERADORES LÓGICOS

&&

Ejecutar si éxito

```
mkdir dir && cd dir
```

||

Ejecutar si fallo

```
cd /no || echo "Error"
```

☰ COMODINES

?

Un carácter cualquiera

```
ls file?.txt
```

*

Múltiples caracteres

```
ls *.txt
```

[]

Rango de caracteres

```
ls [abc]*.txt
```

📄 EJEMPLOS PRÁCTICOS

```
cat /etc/passwd | grep "bash" | cut -d: -f1
cd /var/log && ls -lh *.log
grep "error" /var/log/* 2>/dev/null | head -10
find . -name "*.tmp" -exec rm {} \;
for f in *.txt; do echo "$f"; done
ls -la /home/user/[Dd]ocuments
```

💡 CONSEJOS ÚTILES

- **Pipes encadenados:** combinan múltiples comandos para procesamiento complejo
- **&& vs ;:** && solo ejecuta si el anterior tuvo éxito
- **Comodines:** ? es exactamente un carácter, * es cualquier cantidad
- **[] rangos:** [0-9] números, [a-z] letras minúsculas
- **|| fallback:** ejecuta el segundo comando si el primero falla



EJERCICIO 1. PROCESAMIENTO DE ARCHIVOS DE SISTEMA

✓ OBJETIVO DEL EJERCICIO

Obtener una lista ordenada de usuarios con shell **bash** desde `/etc/passwd`

<> PASOS A SEGUIR

- 1 Leer el archivo `/etc/passwd`
- 2 Filtrar líneas que contengan `"/bash"`
- 3 Extraer solo el campo del nombre de usuario (campo 1)
- 4 Ordenar la lista alfabéticamente

⚠ ARCHIVO DE EJEMPLO

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
user:x:1000:1000:User:/home/user:/bin/bash
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
```

📄 SOLUCIÓN PASO A PASO

Paso 1: Ver contenido del archivo

```
cat /etc/passwd
```

Paso 2: Filtrar usuarios con bash

```
cat /etc/passwd | grep "/bash"
```

Paso 3: Extraer nombre de usuario

```
cat /etc/passwd | grep "/bash" | cut -d: -f1
```

Paso 4: Ordenar la lista

```
cat /etc/passwd | grep "/bash" | cut -d: -f1 | sort
```

✓ RESULTADO ESPERADO

```
root
user
... (otros usuarios con bash)
```

RESUMEN Y CONCLUSIONES

CONCEPTOS CLAVE

- ▶ Sintaxis de comandos
- ▶ Procesamiento de texto
- ▶ Descriptores de ficheros
- ▶ Comodines y operadores
- ▶ Comandos internos/externos
- ▶ Expresiones regulares
- ▶ Redirecciones y pipes
- ▶ Procesamiento complejo

CONSEJOS FINALES

- Practica con archivos del sistema: `/etc/passwd`, `/var/log/*`
- Combina pipes para procesamientos complejos
- Usa `man` y `--help` para aprender más opciones
- Redirige stderr a `/dev/null` para ocultar errores
- Verifica siempre tus comandos antes de ejecutarlos

MEJORES PRÁCTICAS

- ✓ Documenta tus comandos
- ✓ Prueba en entornos seguros
- ✓ Usa scripts para tareas repetitivas
- ✓ Revisa permisos antes de redirigir

RECURSOS PARA APRENDER

-  **Documentación oficial**
man pages, info pages
-  **TLDR Pages**
tldr.sh - Ejemplos prácticos
-  **ShellCheck**
www.shellcheck.net - Analizador de scripts
-  **LinuxCommand.org**
Tutoriales y guías completas
-  **Práctica diaria**
Desafíos en línea como cmdchallenge.com

FLUJO DE TRABAJO RECOMENDADO

- 1 Identificar el objetivo del comando
- 2 Construir paso a paso con pipes
- 3 Probar cada paso por separado
- 4 Agregar redirecciones al final
- 5 Documentar y guardar el comando final